

API Endpoints Specifikation

Komplett Backend-mappning för Onboarding-flödet

Celestial AB - Roaring

22 oktober 2025

Innehåll

1	Introduktion	6
1.1	Dokumentstruktur	6
1.2	Teknisk Stack	6
1.3	Base URL	6
1.4	Autentisering	6
1.5	Autentisering och Session Management - Teori	6
1.5.1	Översikt: JWT-baserad Autentisering	6
1.5.2	Token-typer i vårt system	7
1.5.3	Token-flöde: Komplett Livscykel	9
1.5.4	Säkerhetsaspekter	10
1.5.5	Sammanfattning: Vad lagras var?	11
2	Landing Page (Hero-sektion)	12
2.1	API-endpoints Sammanfattning	12
2.2	Översikt	12
2.3	Funktionalitet	12
2.4	localStorage Check och Auth-flöde	12
2.5	Endpoint 1: GET /api/auth/session	13
2.5.1	Beskrivning	13
2.5.2	Request	13
2.5.3	Response - Success (200 OK - Authenticated)	13
2.5.4	Response - Not Authenticated (200 OK)	13
2.5.5	Implementation Notes	13
2.6	Endpoint 2: POST /api/auth/refresh	13
2.6.1	Beskrivning	13
2.6.2	Request	14
2.6.3	Response - Success (200 OK)	14
2.6.4	Response - Error (401 Unauthorized)	14
2.6.5	Implementation Notes	14
2.7	Frontend - Datalagring i Browser	14
2.7.1	LocalStorage	14
2.7.2	Cookies	15
2.8	Backend - Behandlingshistorik och GDPR-loggning	15
2.8.1	Loggade händelser	15

2.8.2	CSV-filer (Mock Database)	16
2.9	Databastabeller (Framtida PostgreSQL)	16
2.10	Sammanfattning	17
2.10.1	Scenario 1: Ny besökare (okänd användare)	17
2.10.2	Scenario 2: Återkommande kund (sessionen har gått ut)	17
2.10.3	Scenario 3: Inloggad kund (aktiv session)	18
2.10.4	Teknisk sammanfattning	18
3	Registrering (Sida 2)	19
3.1	API-endpoints Sammanfattning	19
3.2	Översikt	19
3.3	Funktionalitet	19
3.4	Endpoint 1: POST /api/auth/register	19
3.4.1	Beskrivning	19
3.4.2	Request	19
3.4.3	Response - Success (201 Created)	19
3.4.4	Response - Error (400 Bad Request)	20
3.4.5	Response - Success (200 OK - Dubblettregistrering)	20
3.4.6	Implementation Notes	20
3.4.7	Databastabeller	21
3.5	Endpoint 2: POST /api/auth/google	21
3.5.1	Beskrivning	21
3.5.2	Request	21
3.5.3	Response - Success (200 OK för befintlig, 201 Created för ny användare)	21
3.5.4	Implementation Notes	22
3.6	Endpoint 3: GET /api/auth/check-email	22
3.6.1	Beskrivning	22
3.6.2	Request	22
3.6.3	Response - Success (200 OK)	22
3.6.4	Implementation Notes	22
3.7	Cloudflare Turnstile Integration	23
3.7.1	Frontend Implementation	23
3.7.2	Backend Validation	23
3.8	Frontend - Datalagring i Browser	23
3.8.1	LocalStorage	23
3.8.2	Cookies	24
3.9	Backend - Behandlingshistorik och GDPR-loggning	24
3.9.1	Loggade händelser	24
3.9.2	CSV-filer (Mock Database)	25
3.10	Databastabeller (Framtida PostgreSQL)	25
3.11	Sammanfattning	26
3.11.1	Scenario 1: Email/Password-registrering (standard-flöde)	26
3.11.2	Scenario 2: Google OAuth-registrering (snabbt flöde)	27
3.11.3	Scenario 3: Dubblettregistrering (edge case)	28
3.11.4	Teknisk sammanfattning	29

4	Kodinmatning Email-verifiering (Sida 3)	30
4.1	API-endpoints Sammanfattning	30
4.2	Översikt	30
4.3	Funktionalitet	30
4.4	Endpoint 1: POST /api/auth/verify-code	30
4.4.1	Beskrivning	30
4.4.2	Request	30
4.4.3	Response - Success (200 OK)	31
4.4.4	Response - Error	31
4.4.5	Implementation Notes	32
4.4.6	Databastabeller	32
4.5	Endpoint 2: POST /api/auth/resend-code	32
4.5.1	Beskrivning	32
4.5.2	Request	32
4.5.3	Response - Success (200 OK)	33
4.5.4	Response - Error	33
4.5.5	Implementation Notes	33
4.6	Frontend - Datalagring i Browser	33
4.6.1	LocalStorage	33
4.6.2	Cookies	34
4.7	Backend - Behandlingshistorik och GDPR-loggning	34
4.7.1	Loggade händelser	34
4.7.2	CSV-filer (Mock Database)	35
4.8	Databastabeller (Framtida PostgreSQL)	35
4.9	Sammanfattning	36
4.9.1	Scenario 1: Lyckad verifiering (happy path)	36
4.9.2	Scenario 2: Koden gått ut, begär ny kod	36
4.9.3	Scenario 3: För många felaktiga försök	37
4.9.4	Teknisk sammanfattning	37
5	Login (Sida 4)	38
5.1	API-endpoints Sammanfattning	38
5.2	Översikt	38
5.3	Funktionalitet	38
5.4	Endpoint 1: POST /api/auth/login	38
5.4.1	Beskrivning	38
5.4.2	Beskrivning	38
5.4.3	Request	38
5.4.4	Response - Success (200 OK)	39
5.4.5	Response - Success (200 OK)	39
5.4.6	Response - Error	39
5.4.7	Implementation Notes	40
5.4.8	Implementation Notes	40
5.4.9	Databastabeller	40
5.5	Endpoint 2: POST /api/auth/google	40
5.5.1	Beskrivning	40
5.5.2	Beskrivning	40
5.5.3	Request	40

5.5.4	Response - Success (200 OK för login, 201 Created för ny användare)	41
5.5.5	Response - Success (200 OK för login, 201 Created för ny användare)	41
5.5.6	Response - Error	41
5.5.7	Implementation Notes	42
5.5.8	Externa API-anrop	42
5.6	Frontend - Datalagring i Browser	42
5.6.1	LocalStorage	42
5.6.2	Cookies	43
5.7	Backend - Behandlingshistorik och GDPR-loggning	43
5.7.1	Loggade händelser	43
5.7.2	CSV-filer (Mock Database)	44
5.8	Databastabeller (Framtida PostgreSQL)	45
5.9	Frontend Implementation (LoginSlide.jsx)	46
5.9.1	Email/Password Login	46
5.9.2	Google OAuth Login	47
5.10	Sammanfattning	48
5.10.1	Scenario 1: Lyckad inloggning med Email/Password	48
5.10.2	Scenario 2: Lyckad inloggning med Google OAuth	49
5.10.3	Scenario 3: Misslyckad inloggning (Fel lösenord)	50
5.10.4	Scenario 4: Inloggning nekad (Overifierad email)	50
5.10.5	Teknisk sammanfattning	50
6	Intro (Slide 5)	52
7	Uppdrag (Slide 6)	53
8	Verksamhet (Slide 7)	54
9	Riskfrågor (Slide 8)	55
10	Identitetskontroll (Slide 9)	56
11	Kontrolltabell (Slide 10)	57
12	PEP-fördjupning (Slide 11)	58
13	Verksamhetskod (Slide 12)	59
14	Ägarstruktur (Slide 13)	60
15	Styrelse (Slide 14)	61
16	Företagsdokumentation (Slide 13.5)	62
17	Bokföringsunderlag (Slide 14.5)	63
18	Bokföring (Slide 15)	64
19	Likviditet (Slide 16)	65
20	Omsättning (Slide 17)	66

21 Resultat (Slide 18)	67
22 Bransch (Slide 19)	68
23 Bokföringsanalys (Slide 20)	69
24 Riskbesked (Slide 21)	70
25 Skyldigheter (Slide 22)	71
26 Fortnox-koppling (Slide 23)	72
27 Bankrättigheter (Slide 24)	73
28 AdminDashboard	74
29 Appendix	75
29.1 Gemensamma Error Codes	75
29.2 Rate Limiting	75
29.3 File Upload Limits	75

1 Introduktion

Detta dokument beskriver **alla API endpoints** som behövs för att implementera den kompletta onboarding-processen enligt `Onboardin_app.ny.tex`.

1.1 Dokumentstruktur

Dokumentet är uppdelat i två huvuddelar:

Del 1: Autentiseringssidor (Utan Sidebar) – Sidor som visas innan användaren är inloggad, inklusive landing page, registrering, verifiering, inloggning och lösenordsåterställning. Dessa sidor har ingen sidebar.

Del 2: Onboarding Slides (Med Sidebar) – Slides som visas efter inloggning och utgör själva onboarding-processen. Dessa har sidebar med progressindikator och navigering.

För varje slide/steg i onboarding-flödet dokumenteras:

- Vilka API endpoints som behövs
- HTTP-metod (GET, POST, PUT, DELETE)
- Request-format (JSON schema, multipart/form-data för filer)
- Response-format (success och error cases)
- Affärslogik och valideringar
- Externa integrationer (Bolagsverket, Skatteverket, BankID, etc.)
- Databastabeller som påverkas

1.2 Teknisk Stack

- **Backend:** FastAPI (Python 3.11+)
- **Databas:** PostgreSQL 15+
- **Authentication:** JWT tokens (access + refresh)
- **File Storage:** AWS S3 eller lokal disk under utveckling
- **OCR:** Google Cloud Vision API eller Tesseract
- **Externa API:er:** Bolagsverket, Skatteverket, BankID, SendGrid

1.3 Base URL

Development: `http://localhost:8000/api`

Production: `https://api.roaring.se/api`

1.4 Autentisering

Alla endpoints (utom registrering, login, verify) kräver JWT token i header:

Authorization: Bearer <access_token>

1.5 Autentisering och Session Management - Teori

1.5.1 Översikt: JWT-baserad Autentisering

Vår applikation använder **JWT (JSON Web Tokens)** för autentisering, INTE traditionella session cookies. Detta innebär att backend är **stateless** – servern behöver inte hålla reda på vilka användare som är inloggade i en sessions-tabell.

Varför JWT istället för session cookies?

- **Skalbarhet:** Ingen server-side session storage behövs (viktigt för mikroservices)
- **Cross-domain:** Fungerar mellan olika domäner/subdomäner
- **Mobile-friendly:** Enklare för mobilappar (ingen cookie-hantering)
- **Decentraliserad:** Tokens kan verifieras av vilken server som helst med samma secret

1.5.2 Token-typer i vårt system

1. Access Token (JWT)

Syfte: Bevisar att användaren är autentiserad och får göra API-requests.

Livslängd: 15 minuter

Lagring: `localStorage.accessToken` (frontend)

Innehåll (JWT payload):

```
{
  "sub": "user_12345",           // User ID
  "email": "lasse@example.com",
  "role": "byrå_admin",
  "firm_id": "firm_67890",
  "iat": 1729600000,             // Issued at (Unix timestamp)
  "exp": 1729600900             // Expires at (15 min senare)
}
```

Användning:

- Skickas med VARJE API-request i header: **Authorization: Bearer <token>**
- Backend verifierar JWT signature och **exp** (expiration time)
- Om giltig: Request godkänns, annars: 401 Unauthorized

Säkerhet:

- Kort livslängd (15 min) begränsar skada om token stjäls
- Signerad med `JWT_SECRET` (backend environment variable)
- Kan INTE ändras av klient utan att signaturen blir ogiltig

2. Refresh Token (JWT)

Syfte: Används för att få nya access tokens utan att användaren måste logga in igen.

Livslängd: 30 dagar

Lagring:

- Frontend: `localStorage.refreshToken`
- Backend: `refresh_tokens.csv` (eller PostgreSQL-tabell) – token hash + user_id + expires_at

Innehåll (JWT payload):

```
{
  "sub": "user_12345",
  "type": "refresh",
  "iat": 1729600000,
  "exp": 1732192000             // 30 dagar senare
}
```

Användning:

- När access token går ut (efter 15 min) → Frontend skickar refresh token till `POST /api/auth/refresh`
- Backend verifierar refresh token (signature + finns i `refresh_tokens.csv`?)
- Om giltig: Generera ny access token (15 min) + ny refresh token (30 dagar)
- Returnera båda till frontend → Frontend uppdaterar `localStorage`

Säkerhet:

- Lagras BÅDE i frontend OCH backend (kan revokeas om behövs)
- Vid lösenordsändring eller logout: Ta bort alla refresh tokens för användaren från backend
- Rotation: Varje gång refresh token används genereras en NY refresh token (gamla invalideras)

3. Session Cookies (`sessionId`) – ANVÄNDS INTE I VÅR APP!

OBS: Även om du kan se cookies med namn som `sessionId` eller liknande i Chrome DevTools, är dessa INTE en del av vår autentiseringslogik.

Varför du kan se `sessionId` cookies ändå:

- **Vite Dev Server:** Skapar ibland egna cookies för HMR (Hot Module Replacement)
- **Cloudflare:** Om du använder Cloudflare framför din app sätter de cookies (t.ex. `__cf_bm`, `cf_clearance`)
- **Andra libraries:** Analytics (Matomo), crash reporting, etc. kan sätta cookies
- **Browser extensions:** Kan injektera cookies

Hur traditionella session cookies fungerar (för jämförelse):

I ett session-baserat system (som vi INTE använder):

1. Användaren loggar in → Backend genererar random `sessionId` (t.ex. `abc123xyz`)
2. Backend sparar i sessions-tabell: `sessionId` → `userId`, `expires_at`
3. Backend skickar cookie: `Set-Cookie: sessionId=abc123xyz; HttpOnly; Secure`
4. Vid varje request: Browser skickar cookie automatiskt → Backend kollar sessions-tabell
5. Vid logout: Backend raderar session från tabell

Varför vi inte använder detta:

- Kräver server-side session storage (databas eller Redis)
- Svårare att skala horisontellt (olika servrar måste dela session-state)
- Mer komplext för SPA (Single Page Applications) som vår React-app

4. Cookie Consent Cookie (`cookie_consent`)

Syfte: Sparar användarens val om cookie-godkännande (GDPR-krav).

Livslängd: 365 dagar

Lagring: Cookie (INTE `HttpOnly`, JavaScript måste kunna läsa den)

Innehåll: `all` (alla cookies) eller `necessary` (endast nödvändiga)

Användning:

- Sätts när användaren klickar "Tillåt alla cookies" eller "Endast nödvändiga cookies"
- Frontend kollar denna cookie vid varje sidladdning

- Om `null`: Visa cookie banner
- Om `necessary`: Ladda INTE analytics/marketing scripts
- Om `all`: Ladda alla scripts (Matomo, Facebook Pixel, etc.)

OBS: Denna cookie är EN AV DE FÅ COOKIES vi faktiskt använder (tillsammans med Cloudflare Turnstile `cf_clearance`).

1.5.3 Token-flöde: Komplet Livscykel

Scenario 1: Registrering och första inloggning

1. Användaren registrerar sig (POST `/api/auth/register`) → Sparas i `aspiring_users.csv` med `verified=false`
2. Backend skickar 6-siffrig kod via SendGrid
3. Användaren matar in kod (POST `/api/auth/verify-code`)
4. Backend:
 - Verifierar kod är korrekt och inte utgången
 - KOPIERAR användare från `aspiring_users.csv` → `users.csv` (med `verified=true`)
 - RADERAR användare från `aspiring_users.csv`
 - Genererar `accessToken` (15 min) + `refreshToken` (30 dagar)
 - Sparar refresh token hash i `refresh_tokens.csv`
5. Backend returnerar: `{accessToken: "...", refreshToken: "...", "user": {...}}`
6. Frontend:
 - Sparar i `localStorage`: `localStorage.setItem(accessToken, ...)`
 - Sparar i `localStorage`: `localStorage.setItem(refreshToken, ...)`
 - Navigerar till `/inledning` (första onboarding-slide)

Scenario 2: Återkommande besökare (Landing Page)

1. Användaren besöker landing page (`/`)
2. Frontend kollar: Finns `localStorage.accessToken`?
3. Om JA:
 - Skicka till backend: GET `/api/auth/session` med `Authorization: Bearer <accessToken>`
 - Backend verifierar JWT signature + exp
 - Om giltig: Returnera `{authenticated: true, "user": {...}}`
 - Frontend: Navigera till `/dashboard` (användaren är inloggad)
4. Om access token utgången (efter 15 min):
 - Backend returnerar 401 Unauthorized
 - Frontend skickar POST `/api/auth/refresh` med `refreshToken` i body
 - Backend verifierar refresh token, genererar NY access token + NY refresh token
 - Frontend uppdaterar `localStorage` och försöker igen
5. Om refresh token också utgången (efter 30 dagar):
 - Backend returnerar 401 Unauthorized
 - Frontend raderar tokens från `localStorage`
 - Visa landing page (användaren måste logga in igen)

Scenario 3: API Request med utgången Access Token

1. Frontend gör request: `GET /api/firms/12345` med access token (som är utgången)
2. Backend returnerar: `401 Unauthorized {"error": token_expired}`
3. Frontend interceptor (axios/fetch middleware) fångar 401:
 - Pausa ursprungliga requesten
 - Skicka `POST /api/auth/refresh` med refresh token
 - Om lyckat: Uppdatera `localStorage` med nya tokens
 - Försök ursprungliga requesten igen med ny access token
4. Om refresh också misslyckas:
 - Radera tokens från `localStorage`
 - Redirect till `/login`

Scenario 4: Logout

1. Användaren klickar Logga ut"
2. Frontend skickar: `POST /api/auth/logout` med refresh token i body
3. Backend:
 - Tar bort refresh token från `refresh_tokens.csv`
 - Loggar händelse i `audit_log.csv`
4. Frontend:
 - Raderar `localStorage.removeItem("accessToken")`
 - Raderar `localStorage.removeItem("refreshToken")`
 - Navigerar till `/`

1.5.4 Säkerhetsaspekter

XSS (Cross-Site Scripting) Skydd:

- JWT tokens i `localStorage` kan läsas av JavaScript (både din kod OCH injected scripts)
- Skydd: Content Security Policy (CSP) headers, sanitize all user input
- Alternativ: Lagra tokens i `HttpOnly` cookies (men svårare för SPA:er)

CSRF (Cross-Site Request Forgery) Skydd:

- JWT i `localStorage` är INTE sårbart för CSRF (skickas manuellt i header, inte automatiskt av browser)
- Session cookies ÄR sårbara (skickas automatiskt) → Kräver CSRF-tokens

Token Rotation:

- Varje gång refresh token används → Generera NY refresh token
- Gamla refresh token invalideras (tas bort från backend)
- Förhindrar replay attacks" (någon stjälar gammal refresh token)

HTTPS:

- **ALLTID** använd HTTPS i produktion
- HTTP skickar tokens i klartext → Lätt att sno med Wireshark

1.5.5 Sammanfattning: Vad lagras var?

Data	Lagringsplats	Livslängd	Syfte
Access Token (JWT)	localStorage (frontend)	15 min	API-autentisering
Refresh Token (JWT)	localStorage (frontend)	30 dagar	Förnya access token
Refresh Token Hash	refresh_tokens.csv (backend)	30 dagar	Verifiering + revokation
cookie_consent	Cookie	365 dagar	GDPR cookie-val
cf_clearance	Cookie (Cloudflare)	30 min	Bot-skydd (Turnstile)
sessionId	ANVÄNDS INTE	N/A	(Traditionell metod)

Viktigt: Om du ser cookies med namn som `sessionId`, `__cf_bm`, eller liknande i Chrome DevTools – dessa är INTE en del av vår autentiseringslogik. De kan komma från Vite dev server, Cloudflare, analytics, eller browser extensions.

2 Landing Page (Hero-sektion)

2.1 API-endpoints Sammanfattning

- GET /api/auth/session — Kontrollera om användaren är inloggad (körs automatiskt om JWT finns i localStorage)
- POST /api/auth/refresh — Förnya access token om den har upphört
- **Antal anrop vid sidladdning:** 0–2 (beroende på om tokens finns i localStorage)

2.2 Översikt

Landing page är den första sidan användaren ser. Den innehåller statiskt innehåll (text, features-lista, CTA-knappar) och kräver **inga API-anrop vid laddning**.

2.3 Funktionalitet

- Visa företagsinformation och värdeproposition
- Knappar: Logga in”, Registrera”, Börja onboarding nu”, Se demo”
- Alla knappar gör frontend-routing (ingen backend-kommunikation)
- **Kontrollera localStorage** för sparade JWT tokens vid sidladdning

2.4 localStorage Check och Auth-flöde

När React-appen laddas (i App.jsx eller auth context) kontrolleras localStorage för JWT tokens:

Steg 1: Kolla localStorage

```
const accessToken = localStorage.getItem('accessToken');  
const refreshToken = localStorage.getItem('refreshToken');
```

Steg 2: Om tokens finns

- Anropa GET /api/auth/session med Authorization: Bearer <accessToken>
- Om 200 OK och authenticated: true → Användaren är inloggad
- Om 401 Unauthorized → Access token har upphört, försök refresh

Steg 3: Om access token upphört

- Anropa POST /api/auth/refresh med refresh token
- Om 200 OK → Spara ny access token, fortsatt som inloggad
- Om 401 → Refresh token upphört, logga ut (rensa localStorage)

Steg 4: Om inga tokens finns

- Användaren är utloggad, visa Logga in” + Registrera”knappar

Knappen Börja onboarding nu”:

- Om användaren HAR giltiga tokens → Frontend-routing till /inledning (ingen API-anrop)
- Om användaren INTE har tokens → Frontend-routing till /login?redirect=/inledning
- Inget dedikerat API-endpoint behövs, bara conditional routing i React

2.5 Endpoint 1: GET /api/auth/session

2.5.1 Beskrivning

Kontrollera om användaren har en aktiv session för att visa rätt UI (inloggad vs utloggad). Anropas automatiskt om JWT token finns i localStorage.

2.5.2 Request

- **Headers:** Authorization: Bearer <token> (optional)
- **Body:** None

2.5.3 Response - Success (200 OK - Authenticated)

```
1 {
2   "authenticated": true,
3   "user": {
4     "id": "550e8400-e29b-41d4-a716-446655440000",
5     "email": "chef@revisionstockholm.se",
6     "role": "byrachef",
7     "firm": {
8       "id": "660e8400-e29b-41d4-a716-446655440001",
9       "name": "Revision Stockholm AB",
10      "orgNr": "556123-4567"
11    }
12  }
13 }
```

2.5.4 Response - Not Authenticated (200 OK)

```
1 {
2   "authenticated": false
3 }
```

2.5.5 Implementation Notes

- Körs automatiskt om access token finns i localStorage
- Token valideras mot JWT secret och expiry time
- Om valid → Query databas för user + firm info
- Om invalid → Returnera **authenticated: false** (ingen DB query)
- Frontend uppdaterar UI baserat på response

2.6 Endpoint 2: POST /api/auth/refresh

2.6.1 Beskrivning

Förnya access token med refresh token om access token har upphört.

2.6.2 Request

```
1 {  
2   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
3 }
```

2.6.3 Response - Success (200 OK)

```
1 {  
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",  
3   "expiresIn": 3600  
4 }
```

2.6.4 Response - Error (401 Unauthorized)

```
1 {  
2   "error": "invalid_token",  
3   "message": "Refresh token har upphört eller är ogiltig"  
4 }
```

2.6.5 Implementation Notes

- Refresh token valideras mot databas (tabell: `refresh_tokens`)
- Kontrollera expiry time (vanligtvis 30 dagar)
- Om valid → Generera ny access token (15 min expiry)
- Om invalid → Användaren måste logga in igen
- Frontend sparar ny access token i localStorage

2.7 Frontend - Datalagring i Browser

2.7.1 LocalStorage

Landing Page kontrollerar localStorage vid laddning för att avgöra om användaren är inloggad:

Nycklar som läses:

- `accessToken` — JWT access token (15 min expiry)
- `refreshToken` — JWT refresh token (30 dagar expiry)

Logik:

- Om båda finns → Anropa GET `/api/auth/session` för validering
- Om access token upphört → Anropa POST `/api/auth/refresh`
- Om inga tokens → Visa Logga in” + Registrera” knappar

Ingen data skrivs till localStorage på denna sida (endast läsning).

2.7.2 Cookies

Landing Page visar en **Cookie Banner** vid första besöket (längst ner på skärmen) för GDPR-compliance.

Cookie Banner (CookieBanner.jsx):

- Visas automatiskt om `localStorage.getItem('cookieConsent')` är `null`
- Position: Fixed bottom, full bredd, z-index 9999
- Text: "Vi använder cookies för att förbättra din upplevelse. Läs vår [Cookie Policy](#)."
- Knappar:
 - **Tillåt alla cookies**" → Sätter `localStorage.setItem('cookieConsent', 'all')`
 - **Tillåt endast nödvändiga cookies**" → Sätter `localStorage.setItem('cookieConsent', 'necessary')`
- Efter val: Banner försvinner och kommer inte tillbaka

Nödvändiga Cookies (sätts utan samtycke):

- **INGA** — Vi använder JWT tokens i `localStorage` istället för session cookies
- Samtycke-valet sparas i `localStorage.cookieConsent`, inte i en cookie

Valfria Cookies (kräver samtycke = "Tillåt alla cookies"):

- **Analytics:** Matomo/Piwik PRO (om implementerat) — `_pk_id`, `_pk_ses`
- **Cloudflare Turnstile:** `cf_clearance` (sätts vid CAPTCHA-validering)
- **Marketing:** INGA i nuvarande version (framtida: LinkedIn, Google Ads)

OBS: JWT tokens (`accessToken`, `refreshToken`) sparas i **localStorage**, inte cookies, så de omfattas EJ av GDPR cookie-regler.

2.8 Backend - Behandlingshistorik och GDPR-loggning

2.8.1 Loggade händelser

Landing Page själv genererar **inga loggposter** då den är statisk. Däremot loggas följande:

1. Cookie-samtycke (när användaren klickar på banner):

Datum: 2025-10-22 18:25:10
Användare: Anonym (ID: NULL)
Program: GDPR Compliance
Rubrik: Cookie Consent
Händelse: Användaren valde "Tillåt alla cookies"
– IP-adress: 81.224.233.137
– User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
– Samtycke: all
– Endpoint: POST /api/gdpr/cookie-consent (frontend-only, inget API-anrop)

OBS: Cookie-samtycke sparas **ENDAST** i `localStorage` (frontend), inte på servern. Loggning sker bara om vi senare implementerar server-side tracking av GDPR-samtycken för statistik.

2. Vid GET /api/auth/session (om token finns):

Datum: 2025-10-22 18:30:28
Användare: Anders Svensson (ID: 42)
Program: Authentication
Rubrik: Session Check
Händelse: Session validerad
- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: GET /api/auth/session
- Status: 200 OK

3. Vid POST /api/auth/refresh (om access token upphört):

Datum: 2025-10-22 18:31:15
Användare: Anders Svensson (ID: 42)
Program: Authentication
Rubrik: Token Refresh
Händelse: Access token förnyad
- IP-adress: 81.224.233.137
- Refresh token ID: rt_8f9a7b6c...
- Ny access token expiry: 2025-10-22 18:46:15

2.8.2 CSV-filer (Mock Database)

Landing Page interagerar med följande CSV-filer:

data/audit_log.csv:

```
timestamp,user_id,user_name,program,event_type,event_details,ip_address,user_agent
2025-10-22 18:30:28,42,"Anders Svensson",Authentication,"Session Check",
"Session validerad - GET /api/auth/session - 200 OK",81.224.233.137,
"Mozilla/5.0 (X11; Linux x86_64)..."
```

data/users.csv (läses vid session check):

```
id,email,password_hash,role,firm_id,verified,created_at
42,anders@revisionstockholm.se,$2b$12$...,byrachef,7,true,2025-10-15 14:23:10
```

data/refresh_tokens.csv (läses vid token refresh):

```
id,user_id,token_hash,expires_at,created_at
rt_8f9a7b6c...,42,$2b$12$...,2025-11-21 18:30:28,2025-10-22 18:30:28
```

2.9 Databastabeller (Framtida PostgreSQL)

När vi migrerar från CSV till PostgreSQL kommer följande tabeller användas:

audit_log:

```
1 CREATE TABLE audit_log (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   timestamp TIMESTAMP NOT NULL DEFAULT NOW(),
4   user_id UUID REFERENCES users(id),
5   user_name VARCHAR(255),
6   program VARCHAR(100), -- 'Authentication', 'Onboarding', etc.
```



```

7   event_type VARCHAR(100), -- 'Session Check', 'Token Refresh',
   etc.
8   event_details TEXT,
9   ip_address INET,
10  user_agent TEXT,
11  INDEX idx_user_id (user_id),
12  INDEX idx_timestamp (timestamp),
13  INDEX idx_event_type (event_type)
14 );

```

users och **refresh_tokens** (se Login Sida 4 för fullständiga scheman).

2.10 Sammanfattning

2.10.1 Scenario 1: Ny besökare (okänd användare)

En potentiell kund besöker hemsidan för första gången. Webbläsaren har inga sparade tokens i localStorage. Landing Page visar därför standardvyn med knappar för Logga in och Registrera dig”.

Cookie Banner visas: Om `localStorage.getItem('cookieConsent')` är null, visas CookieBanner.jsx längst ner på skärmen med två knappar:

- Tillåt alla cookies” → Sparar `cookieConsent: 'all'` i localStorage
- Tillåt endast nödvändiga cookies” → Sparar `cookieConsent: 'necessary'` i localStorage

Inga API-anrop görs till servern - sidan är helt statisk. Om besökaren klickar Börja onboarding nmdirigeras hen till inloggningssidan med en redirect-parameter som pekar tillbaka till onboarding-flödet.

Backend-loggning: Ingen. Servern vet inte om att besökaren existerar. (Cookie-samtycke sparas endast i localStorage, ingen server-kommunikation.)

2.10.2 Scenario 2: Återkommande kund (sessionen har gått ut)

En befintlig kund, Anders Svensson på Revision Stockholm AB, besöker hemsidan efter 2 veckor. Hans `accessToken` (15 min expiry) har naturligtvis gått ut, men hans `refreshToken` (30 dagar expiry) finns fortfarande kvar i localStorage.

Vad händer:

1. Landing Page läser localStorage och hittar båda tokens
2. Anropar `GET /api/auth/session` med den utgångna access token
3. Servern svarar 401 Unauthorized (Token har upphört”)
4. Frontend anropar automatiskt `POST /api/auth/refresh` med refresh token
5. Servern validerar refresh token mot `refresh_tokens.csv`, genererar ny access token
6. Frontend sparar nya access token i localStorage
7. Landing Page visar nu personlig vy: ”Välkommen tillbaka, Anders!med knapp FF-ortsätt onboarding”

Backend-loggning: Två poster i `audit_log.csv`:

- Session Check (401 - Token expired)
- Token Refresh (200 - Ny access token utfärdad)

2.10.3 Scenario 3: Inloggad kund (aktiv session)

Samma Anders Svensson besöker hemsidan 5 minuter efter att ha loggat in (access token fortfarande giltig).

Vad händer:

1. Landing Page läser localStorage och hittar giltig access token
2. Anropar `GET /api/auth/session`
3. Servern validerar token, läser `users.csv` och returnerar användarinfo
4. Landing Page visar personlig vy direkt: "Välkommen tillbaka, Anders!"

Backend-loggning: En post i `audit_log.csv`:

- Session Check (200 - Session validerad) med IP-adress och User-Agent

2.10.4 Teknisk sammanfattning

- **API-endpoints:** `GET /api/auth/session`, `POST /api/auth/refresh`
- **Antal API-anrop:** 0 (ny besökare), 1 (aktiv session), 2 (utgången session)
- **CSV-filer:** `audit_log.csv`, `users.csv`, `refresh_tokens.csv`
- **Frontend-knappar:** Alla gör routing (inga API-anrop)

3 Registrering (Sida 2)

3.1 API-endpoints Sammanfattning

- POST /api/auth/register — Skapa nytt konto med email/lösenord, skicka 6-siffrig kod
- POST /api/auth/google — Registrera eller logga in via Google OAuth
- GET /api/auth/check-email — Real-time validering om email redan är registrerad
- **Externa integrationer:** Cloudflare Turnstile (bot-skydd), SendGrid (email), Google OAuth

3.2 Översikt

Registreringssidan låter användaren skapa ett nytt konto med email + lösenord eller via Google OAuth. Ett bot-skydd (Cloudflare Turnstile) används för att förhindra automatiserad spam.

3.3 Funktionalitet

- Formulär: Email, Lösenord, Bekräfta lösenord
- Lösenordskrav: Minst 12 tecken, en siffra, en versal
- Cloudflare Turnstile checkbox (bot-skydd)
- Alternativ: Fortsätt med Google” (OAuth)
- Real-time validering av email (om redan registrerad)
- Vid lyckad registrering → Redirect till /verify med email i state

3.4 Endpoint 1: POST /api/auth/register

3.4.1 Beskrivning

Skapa nytt användarkonto, generera 6-siffrig verifieringskod, skicka via email.

3.4.2 Request

```
1 {  
2   "email": "chef@revisionstockholm.se",  
3   "password": "SecurePass123!",  
4   "passwordConfirm": "SecurePass123!",  
5   "captchaToken": "cloudflare-turnstile-response-token"  
6 }
```

3.4.3 Response - Success (201 Created)

```
1 {  
2   "success": true,  
3   "userId": "550e8400-e29b-41d4-a716-446655440000",  
4   "message": "Konto skapat. En 6-siffrig verifieringskod har  
5     skickats till din e-post.",  
6   "email": "chef@revisionstockholm.se"
```

```
6 }
```

3.4.4 Response - Error (400 Bad Request)

Email redan registrerad:

```
1 {
2   "error": "email_exists",
3   "message": "Email-adressen ar redan registrerad"
4 }
```

Svagt lösenord:

```
1 {
2   "error": "weak_password",
3   "message": "Lösenordet maste vara minst 12 tecken, innehalla en
4             siffra och en versal"
```

Captcha misslyckades:

```
1 {
2   "error": "captcha_failed",
3   "message": "Bot-skyddet misslyckades. Forsok igen."
4 }
```

3.4.5 Response - Success (200 OK - Dubblettregistrering)

Email finns redan i aspiring_users.csv (mindre än 24h):

```
1 {
2   "success": true,
3   "userId": "au_abc123",
4   "message": "Ett konto med denna e-post vantar redan pa
5             verifiering. En ny verifieringskod har skickats till din e-
6             post.",
7   "email": "chef@revisionstockholm.se",
8   "isResend": true
9 }
```

OBS: Detta är INTE ett fel - användaren får ny kod utan att skapa duplicat i aspiring_users.csv.

3.4.6 Implementation Notes

- Validera email format (regex)
- **Dubbletthantering - Steg 1:** Kontrollera om email finns i users.csv
 - Om JA → Returnera 400 "Email redan registrerad" (avvisa)
- **Dubbletthantering - Steg 2:** Kontrollera om email finns i aspiring_users.csv
 - Om JA och created_at mindre än 24h sedan:
 - * Generera NY verifieringskod, spara i verification_codes.csv

- * Skicka email med ny kod via SendGrid
- * Returnera 200 "En ny verifieringskod har skickats" (INGEN ny rad i `aspiring_users`!)
- * Logga Dubblettregistrering - Ny kod skickad till `audit_log.csv`
- * EXIT (hoppa över steg nedan)
- Om JA och `created_at` äldre än 24h:
 - * Radera gamla raden från `aspiring_users.csv`
 - * Radera gamla verifieringskoder från `verification_codes.csv`
 - * Fortsätt till normal registrering nedan
- Validera lösenord: minst 12 tecken, `/[A-Z]/`, `/[0-9]/`
- Kontrollera `password === passwordConfirm`
- Validera Cloudflare Turnstile token mot Cloudflare API
- Hasha lösenord med `bcrypt` (cost factor 12)
- Skapa rad i `aspiring_users.csv` med `expires_at = nu + 7 dagar`
- Generera 6-siffrig kod (t.ex. 482916)
- Spara kod i `verification_codes.csv` med 15 min expiry
- Skicka email via SendGrid: *Din verifieringskod är: 482916. Koden är giltig i 15 minuter.*
- Logga till `audit_log.csv`
- Returnera 201 Created
- Frontend navigerar till `/verify` med email i state

3.4.7 Databastabeller

- `users` (id, email, password_hash, verified, created_at, updated_at)
- `verification_codes` (id, email, code, expires_at, attempts, created_at)

3.5 Endpoint 2: POST `/api/auth/google`

3.5.1 Beskrivning

Logga in eller registrera med Google OAuth (knapptext: Fortsätt med Google). Ingen email-verifiering behövs (Google har redan verifierat).

3.5.2 Request

```

1 {
2   "code": "google-oauth-authorization-code",
3   "redirectUri": "http://localhost:5173/auth/callback"
4 }
```

3.5.3 Response - Success (200 OK för befintlig, 201 Created för ny användare)

```

1 {
2   "success": true,
3   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
4   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
5   "user": {
```

```

6   "id": "550e8400-e29b-41d4-a716-446655440000",
7   "email": "chef@gmail.com",
8   "verified": true,
9   "authProvider": "google"
10 },
11 "isNewUser": false
12 }

```

3.5.4 Implementation Notes

- Validera OAuth code mot Google Token API
- Hämta user info (email, name) från Google
- Kontrollera om user finns i databas (sök på email)
- Om finns: Logga in, generera JWT tokens, returnera `isNewUser: false`
- Om ny: Skapa user med `verified: true, auth_provider: 'google', password_hash: null`, returnera `isNewUser: true`
- Generera access token (15 min expiry) och refresh token (30 dagar)
- Spara refresh token i `refresh_tokens` tabell
- Frontend sparar tokens i `localStorage` och navigerar till `/inledning`

3.6 Endpoint 3: GET `/api/auth/check-email`

3.6.1 Beskrivning

Real-time validering om email redan är registrerad (när användare skriver i formuläret).

3.6.2 Request

GET `/api/auth/check-email?email=chef@revision.se`

3.6.3 Response - Success (200 OK)

Tillgänglig:

```

1 {
2   "available": true
3 }

```

Upptagen:

```

1 {
2   "available": false,
3   "message": "Email redan registrerad"
4 }

```

3.6.4 Implementation Notes

- Rate limit: 10 requests/minut per IP
- Query users tabell: `SELECT id FROM users WHERE email = ?`
- Om finns: `available: false`
- Används för UX (visa rött meddelande under input-fältet)

3.7 Cloudflare Turnstile Integration

3.7.1 Frontend Implementation

1. Ladda Cloudflare Turnstile script:

```
<script src="https://challenges.cloudflare.com/turnstile/v0/api.js"
      async defer></script>
```

2. Rendera widget i React:

```
1 <div className="cf-turnstile"
2   data-sitekey="din-site-key-har"
3   data-callback="onTurnstileSuccess"></div>
```

3. Hantera callback:

```
1 const onTurnstileSuccess = (token) => {
2   setCaptchaToken(token);
3 };
```

4. Skicka token i POST /api/auth/register request

3.7.2 Backend Validation

```
1 import requests
2
3 def verify_turnstile(token: str, remote_ip: str) -> bool:
4     response = requests.post(
5         'https://challenges.cloudflare.com/turnstile/v0/
6         siteverify',
7         json={
8             'secret': 'din-secret-key',
9             'response': token,
10            'remoteip': remote_ip
11        }
12    )
13    result = response.json()
14    return result.get('success', False)
```

3.8 Frontend - Datalagring i Browser

3.8.1 LocalStorage

Registreringssidan **skriver INTE** till localStorage vid registrering. Efter lyckad registrering navigeras användaren till /verify där email-adress skickas som route state (inte sparad i localStorage).

Vid Google OAuth-registrering:

- **accessToken** — JWT access token (15 min expiry)
- **refreshToken** — JWT refresh token (30 dagar expiry)
- Google OAuth ger omedelbar inloggning → tokens sparas direkt

Ingen data skrivs för email/password-registrering (användaren måste verifiera först).

3.8.2 Cookies

Registreringssidan sätter INGA egna cookies. Cloudflare Turnstile kan sätta:

- `cf_clearance` — Cloudflare bot-skydd cookie (sätts automatiskt av Turnstile widget)
- `HttpOnly`, `Secure`, `SameSite=Strict`
- Expiry: 30 minuter (session-baserad)

3.9 Backend - Behandlingshistorik och GDPR-loggning

3.9.1 Loggade händelser

1. Ny registrering (email/password):

Datum: 2025-10-22 19:15:42

Användare: chef@revisionstockholm.se (ID: NULL - ännu ej verifierad)

Program: Authentication

Rubrik: Registrering

Händelse: Nytt konto skapat (overifierat)

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: POST /api/auth/register
- Status: 201 Created
- Email: chef@revisionstockholm.se
- Verifieringskod skickad: Ja (SendGrid)

2. Google OAuth-registrering/login:

Datum: 2025-10-22 19:18:30

Användare: anders@gmail.com (ID: 550e8400...)

Program: Authentication

Rubrik: Google OAuth

Händelse: Ny användare via Google OAuth

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: POST /api/auth/google
- Status: 201 Created
- Auth Provider: google
- Verified: true (automatiskt)

3. Email-tillgänglighetskontroll:

Datum: 2025-10-22 19:15:15

Användare: Anonym (ID: NULL)

Program: Authentication

Rubrik: Email Check

Händelse: Real-time validering

- IP-adress: 81.224.233.137
- Endpoint: GET /api/auth/check-email?email=chef@revision.se
- Status: 200 OK
- Resultat: available=false (email upptagen)

4. Cloudflare Turnstile-validering:

Datum: 2025-10-22 19:15:40
Användare: Anonym (ID: NULL)
Program: Security
Rubrik: Bot Protection
Händelse: Cloudflare Turnstile validerad
- IP-adress: 81.224.233.137
- Turnstile Token: 0.XXXXXX...
- Validering: success=true

3.9.2 CSV-filer (Mock Database)

Registrering interagerar med följande CSV-filer:

data/aspiring_users.csv (temporär - överifierade användare):

```
id,email,password_hash,created_at,expires_at
au_8f9a7b6c,chef@revisionstockholm.se,$2b$12$...,2025-10-22 19:15:42,2025-10-29 19:15
```

OBS: Överifierade användare raderas automatiskt efter 7 dagar för GDPR-compliance.

data/verification_codes.csv:

```
id,email,code,expires_at,attempts,created_at
vc_482a91,chef@revisionstockholm.se,482916,2025-10-22 19:30:42,0,2025-10-22 19:15:42
```

data/users.csv (permanent - endast verifierade):

```
id,email,password_hash,role,firm_id,verified,auth_provider,created_at
550e8400,anders@gmail.com,NULL,byrachef,NULL,true,google,2025-10-22 19:18:30
```

data/audit_log.csv:

```
timestamp,user_id,user_name,program,event_type,event_details,ip_address,user_agent
2025-10-22 19:15:42,NULL,"chef@revisionstockholm.se",Authentication,Registrering,
"Nytt konto skapat (överifierat) - POST /api/auth/register - 201",
81.224.233.137,"Mozilla/5.0..."
```

OBS - Förenklad CSV-strategi:

- **aspiring_users.csv** → Temporär lagring vid registrering (**verified: false**)
- **users.csv** → Permanent lagring efter email-verifiering (**verified: true**)
- Vid verifiering: Kopiera rad från **aspiring_users** → **users**, radera från **aspiring_users**
- Ingen "UPDATE-operation behövs i CSV → Enklare kod!

3.10 Databastabeller (Framtida PostgreSQL)

När vi migrerar från CSV till PostgreSQL kommer följande tabeller användas:

aspiring_users (temporär tabell för överifierade):

```
1 CREATE TABLE aspiring_users (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   email VARCHAR(255) UNIQUE NOT NULL,
4   password_hash TEXT NOT NULL,
```

```

5  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
6  expires_at TIMESTAMP NOT NULL, -- 7 dagar framåt
7  INDEX idx_email (email),
8  INDEX idx_expires_at (expires_at)
9 );

```

verification_codes:

```

1 CREATE TABLE verification_codes (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   email VARCHAR(255) NOT NULL,
4   code CHAR(6) NOT NULL, -- 6-siffrig kod
5   expires_at TIMESTAMP NOT NULL, -- 15 min framåt
6   attempts INT DEFAULT 0,
7   created_at TIMESTAMP NOT NULL DEFAULT NOW(),
8   INDEX idx_email (email),
9   INDEX idx_expires_at (expires_at)
10 );

```

users (permanent tabell):

```

1 CREATE TABLE users (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   email VARCHAR(255) UNIQUE NOT NULL,
4   password_hash TEXT, -- NULL for Google OAuth users
5   role VARCHAR(50) DEFAULT 'byrachef',
6   firm_id UUID REFERENCES firms(id),
7   verified BOOLEAN DEFAULT false,
8   auth_provider VARCHAR(20), -- 'email' or 'google'
9   created_at TIMESTAMP NOT NULL DEFAULT NOW(),
10  updated_at TIMESTAMP NOT NULL DEFAULT NOW(),
11  INDEX idx_email (email),
12  INDEX idx_firm_id (firm_id)
13 );

```

audit_log (se Landing Page Sida 1 för fullständigt schema).

3.11 Sammanfattning

3.11.1 Scenario 1: Email/Password-registrering (standard-flöde)

En ny redovisningsbyrå-chef, Anders Svensson, vill skapa konto. Han besöker `/register`, fyller i email (`anders@revisionstockholm.se`), lösenord (`SecurePass123!`), bekräftar lösenord, och klickar i Cloudflare Turnstile-boxen.

Vad händer:

1. Frontend validerar formulär lokalt (lösenordskrav, matching)
2. Real-time check: `GET /api/auth/check-email` → `available: true`
3. Cloudflare Turnstile widget genererar token (bot-skydd)
4. Anders klickar "Skapa konto"
5. Frontend anropar `POST /api/auth/register`
6. Backend:
 - Validerar Turnstile token mot Cloudflare API → `success: true`

- Hashar lösenord med bcrypt (cost factor 12)
- Skapar rad i `aspiring_users.csv` (INTE `users.csv` ännu!)
- Genererar 6-siffrig kod (t.ex. 482916), sparar i `verification_codes.csv`
- Skickar email via SendGrid: *Din verifieringskod är: 482916. Giltig i 15 min.*
- Loggar händelse till `audit_log.csv`
- Returnerar 201 Created

7. Frontend navigerar till `/verify` med email i route state

Backend-loggning: 4 poster i `audit_log.csv` (Email Check, Turnstile Validation, Registrering, Email Sent).

CSV-filer påverkade:

- `aspiring_users.csv` — +1 rad (overifierad användare)
- `verification_codes.csv` — +1 rad (6-siffrig kod, 15 min expiry)
- `audit_log.csv` — +4 rader

3.11.2 Scenario 2: Google OAuth-registrering (snabbt flöde)

Samma Anders Svensson vill istället registrera sig snabbt via Google. Han klickar Fortsätt med Google på `/register`.

Vad händer:

1. Frontend öppnar Google OAuth popup
2. Anders loggar in med sitt Gmail-konto, godkänner åtkomst
3. Google returnerar authorization code
4. Frontend anropar `POST /api/auth/google` med code
5. Backend:
 - Validerar code mot Google Token API → får access token
 - Hämtar user info från Google: email, name
 - Kontrollerar om email finns i `users.csv` → NEJ (ny användare)
 - Skapar rad DIREKT i `users.csv` (inte `aspiring-users`!)
 - Sätter `verified: true`, `auth_provider: 'google'`, `password_hash: NULL`
 - Genererar JWT tokens (accessToken 15 min, refreshToken 30 dagar)
 - Sparar refreshToken i `refresh_tokens.csv`
 - Loggar till `audit_log.csv`
 - Returnerar 201 Created med tokens
6. Frontend sparar tokens i localStorage
7. Navigerar till `/inledning` (direktinloggad!)

Backend-loggning: 2 poster (Google OAuth, Token Issued).

CSV-filer påverkade:

- `users.csv` — +1 rad (direkt verifierad via Google)
- `refresh_tokens.csv` — +1 rad
- `audit_log.csv` — +2 rader

OBS: Ingen `aspiring_users.csv`, ingen `verification_codes.csv` → Google-användare hoppar över email-verifiering!

3.11.3 Scenario 3: Dubblettregistrering (edge case)

Anders Svensson registrerar sig Dag 1 kl 19:00 men glömmer verifiera. Dag 2 kl 10:00 tror han att han inte registrerade sig igår och försöker registrera sig ****igen**** med samma email.

Vad händer:

1. **Dag 1, 19:00:** Anders registrerar `anders@revision.se`
 - Backend skapar rad i `aspiring_users.csv` (id: `au_abc123`, `expires_at`: 2025-10-08 19:00)
 - Skickar verifieringskod 482916 via email (`expires_at`: 19:15)
2. **Dag 1, 19:15:** Verifieringskod går ut (15 min expiry)
3. **Dag 1, 20:00:** Anders glömmer hela saken
4. **Dag 2, 10:00:** Anders försöker registrera sig igen (samma email!)
5. Backend logik i `POST /api/auth/register`:
 - Kontrollerar `users.csv` → Email finns EJ (OK, fortsatt)
 - Kontrollerar `aspiring_users.csv` → Email FINNS redan! (`au_abc123`)
 - Kontrollerar `created_at`: 2025-10-01 19:00 → 15 timmar sedan (mindre än 24h)
 - **Beslut:** Skicka ny verifieringskod, SKAPA INTE ny rad
 - Genererar ny kod: 739284 (ny rad i `verification_codes.csv`)
 - Returnerar: 200 OK med meddelande: *"Ett konto med denna email väntar redan på verifiering. En ny kod har skickats till din e-post."*
6. Anders får ny email med kod 739284
7. Anders verifierar inom 15 min → Flyttas från `aspiring_users.csv` till `users.csv`

Backend-loggning: 3 poster (Dubblettregistrering-försök, Ny kod skickad, Verifiering lyckades).

CSV-filer påverkade:

- `aspiring_users.csv` — INGEN ny rad (duplicates förhindrade)
- `verification_codes.csv` — +1 rad (ny kod, gammal kod raderas eller expires)
- `audit_log.csv` — +3 rader

Edge case: Om äldre än 24h

Om Anders väntar 25 timmar innan dubblettregistrering:

- Backend raderar gamla raden från `aspiring_users.csv` (`au_abc123`)
- Raderar gamla verifieringskoder från `verification_codes.csv`
- Skapar NY rad i `aspiring_users.csv` (`au_xyz789`)
- Skickar ny verifieringskod
- Returnerar 201 Created (nytt försök, gammal data rensad)

Sammanfattning av dubbletthantering:

- **Om email i `users.csv`:** → 400 "Email redan registrerad" (avvisa)
- **Om email i `aspiring_users.csv` (mindre än 24h):** → 200 Ny kod skickad" (ingen ny rad)
- **Om email i `aspiring_users.csv` (äldre än 24h):** → Radera gammal, skapa ny rad, 201 Created
- **Result:** Ingen risk för dubletter i `aspiring_users.csv`!

3.11.4 Teknisk sammanfattning

- **CSV-strategi:** aspiring_users.csv (temp) → users.csv (permanent vid verifiering)
- **Dubblatthantering:** Email-check i BÅDA users.csv OCH aspiring_users.csv innan insert
- **3 API endpoints:** register, google, check-email
- **Bot-skydd:** Cloudflare Turnstile (validates before registration)
- **Verifiering:** 6-siffrig kod via SendGrid (15 min expiry, max 5 attempts)
- **OAuth:** Google login/registrering (hoppas över email-verifiering, direkt till users.csv)
- **Externa integrationer:** Cloudflare API, Google OAuth API, SendGrid
- **Logging:** Alla registreringar loggas till audit_log.csv (IP, User-Agent, timestamp)
- **GDPR:** Overifierade aspiring_users raderas efter 7 dagar automatiskt
- **Cleanup-jobb:** Radera aspiring_users äldre än 24h vid dubblettregistrering

4 Kodinmatning Email-verifiering (Sida 3)

4.1 API-endpoints Sammanfattning

- POST /api/auth/verify-code — Verifiera 6-siffrig kod, flytta aspiring_user → users.csv, returnera JWT tokens
- POST /api/auth/resent-code — Skicka ny 6-siffrig kod om tidigare gått ut
- **Antal anrop vid lyckad verifiering:** 1 (POST /api/auth/verify-code)
- **Rate limiting:** Max 5 verifieringsförsök per kod, Max 3 resend-requests per 15 minuter
- **Externa integrationer:** SendGrid (email för resend)

4.2 Översikt

Email-verifieringssidan visas direkt efter registrering (Sida 2). Användaren ser 6 input-fält (en siffra i varje) där de matar in den 6-siffriga koden som skickades till deras email. Vid lyckad verifiering flyttas användaren från `aspiring_users.csv` till `users.csv`, JWT tokens utfärdas, och användaren loggas in automatiskt.

4.3 Funktionalitet

- 6 input-fält för kod (auto-fokus nästa fält vid inmatning)
- Real-time validering (endast siffror 0-9)
- Knapp: "Verifiera" → POST /api/auth/verify-code
- Knapp: "Skicka ny kod" → POST /api/auth/resent-code
- Timer: Visar återstående tid (15 min från kodgenerering)
- Felmeddelanden: "Fel kod", "Koden har gått ut", "För många försök"
- Vid lyckad verifiering:
 - Backend returnerar JWT tokens (accessToken, refreshToken)
 - Frontend sparar tokens i localStorage
 - Navigerar till /inledning (första onboarding-slide)

4.4 Endpoint 1: POST /api/auth/verify-code

4.4.1 Beskrivning

Verifiera 6-siffrig kod. Vid success flyttas användaren från `aspiring_users.csv` till `users.csv` med `verified: true`, JWT tokens genereras, och användaren loggas in direkt.

4.4.2 Request

```
1 {  
2   "email": "chef@revisionstockholm.se",  
3   "code": "482916"  
4 }
```

Validering:

- Email finns i `aspiring_users.csv` (annars 404 "User not found")
- Kod finns i `verification_codes.csv` för denna email

- Kod har inte gått ut (`expires_at` < NOW, annars 410 "Code expired")
- Kod matchar (6 siffror exakt, annars 400 "Invalid code")
- Max 5 attempts inte överskriden (annars 429 Too many attempts")

4.4.3 Response - Success (200 OK)

```

1 {
2   "success": true,
3   "message": "Email verifierad! Loggar in...",
4   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
5   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
6   "user": {
7     "id": "550e8400-e29b-41d4-a716-446655440000",
8     "email": "chef@revisionstockholm.se",
9     "verified": true,
10    "role": "byrachef",
11    "firmId": null
12  }
13 }
```

4.4.4 Response - Error

400 Bad Request - Fel kod:

```

1 {
2   "error": "invalid_code",
3   "message": "Fel verifieringskod. Försök igen.",
4   "attemptsRemaining": 3
5 }
```

404 Not Found - Email finns inte i `aspiring_users`:

```

1 {
2   "error": "user_not_found",
3   "message": "Ingen overifierad användare med denna email."
4 }
```

410 Gone - Koden har gått ut:

```

1 {
2   "error": "code_expired",
3   "message": "Verifieringskoden har gått ut. Klicka 'Skicka ny kod'."
4 }
```

429 Too Many Requests - För många försök:

```

1 {
2   "error": "too_many_attempts",
3   "message": "Du har överskridit max antal försök (5). Begär en ny kod."
4 }
```

4.4.5 Implementation Notes

- Validera kod mot `verification_codes.csv` (email, code, expires_at)
- Öka `attempts` counter i `verification_codes.csv` vid varje försök
- Om `attempts` ≥ 5 → Returnera 429 (användaren måste begära ny kod)
- Om `expires_at` $\leq$ NOW → Returnera 410 (koden gått ut)
- Om kod matchar:
 - **Steg 1:** Läs rad från `aspiring_users.csv` (email, password_hash, id)
 - **Steg 2:** Skapa rad i `users.csv` med samma data + `verified: true`
 - **Steg 3:** Radera rad från `aspiring_users.csv`
 - **Steg 4:** Radera verifieringskod från `verification_codes.csv`
 - **Steg 5:** Generera JWT tokens (accessToken 15 min, refreshToken 30 dagar)
 - **Steg 6:** Spara refreshToken i `refresh_tokens.csv`
 - **Steg 7:** Logga händelse till `audit_log.csv`
 - **Steg 8:** Returnera 200 OK med tokens och user info
- Frontend sparar tokens i localStorage
- Frontend navigerar till `/inledning`

4.4.6 Databastabeller

Läses:

- `aspiring_users.csv` (email, password_hash, id, created_at)
- `verification_codes.csv` (email, code, expires_at, attempts)

Skrivs:

- `users.csv` — +1 rad (kopierad från `aspiring_users` + `verified: true`)
- `refresh_tokens.csv` — +1 rad (ny refresh token)
- `audit_log.csv` — +1 rad (Email Verified)

Raderas:

- `aspiring_users.csv` — -1 rad (användaren flyttad till `users.csv`)
- `verification_codes.csv` — -1 rad (koden använd)

4.5 Endpoint 2: POST `/api/auth/resent-code`

4.5.1 Beskrivning

Generera och skicka ny 6-siffrig verifieringskod om tidigare kod gått ut eller tappats bort.

4.5.2 Request

```
1 {  
2   "email": "chef@revisionstockholm.se"  
3 }
```


4.5.3 Response - Success (200 OK)

```
1 {
2   "success": true,
3   "message": "En ny verifieringskod har skickats till din e-post",
4   "expiresIn": 900
5 }
```

OBS: expiresIn är i sekunder (900 = 15 minuter).

4.5.4 Response - Error

404 Not Found - Email finns inte i aspiring_users:

```
1 {
2   "error": "user_not_found",
3   "message": "Ingen overifierad användare med denna email."
4 }
```

429 Too Many Requests - Rate limit:

```
1 {
2   "error": "rate_limit_exceeded",
3   "message": "Du kan bara begära ny kod 3 gånger per 15 minuter.
4   Försök igen om 8 minuter.",
5   "retryAfter": 480
6 }
```

OBS: retryAfter är i sekunder.

4.5.5 Implementation Notes

- Kontrollera att email finns i `aspiring_users.csv`
- Rate limiting: Max 3 resend-requests per 15 minuter (per email)
 - Spåra i `resend_log.csv` eller räkna rader i `verification_codes.csv`
 - Om 3+ koder genererade senaste 15 min → 429 Too Many Requests
- Radera gamla verifieringskoder för denna email från `verification_codes.csv`
- Generera ny 6-siffrig kod (t.ex. 739284)
- Spara ny kod i `verification_codes.csv` med `expires_at = NOW + 15 min`
- Skicka email via SendGrid: *Din nya verifieringskod är: 739284. Giltig i 15 minuter.*
- Logga till `audit_log.csv` (Resend Code Request)
- Returnera 200 OK

4.6 Frontend - Datalagring i Browser

4.6.1 LocalStorage

Email-verifieringssidan SKRIVER till localStorage VID LYCKAD VERIFIERING:

Efter POST /api/auth/verify-code (200 OK):

- `accessToken` — JWT access token (15 min expiry)

- `refreshToken` — JWT refresh token (30 dagar expiry)

Före verifiering:

- Email-adress skickas som route state från `/register` (inte sparad i `localStorage`)
- Ingen data skrivs till `localStorage` under kodmatning

4.6.2 Cookies

Inga cookies sätts på denna sida.

4.7 Backend - Behandlingshistorik och GDPR-loggning

4.7.1 Loggade händelser

1. Lyckad verifiering:

Datum: 2025-10-22 19:20:15

Användare: `chef@revisionstockholm.se` (ID: 550e8400...)

Program: Authentication

Rubrik: Email Verified

Händelse: Email verifierad, användare aktiverad

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: POST `/api/auth/verify-code`
- Status: 200 OK
- Kod: 482916 (matchad)
- Flyttad från `aspiring_users.csv` → `users.csv`
- JWT tokens utfärdade

2. Fel kod (attempt counter ökas):

Datum: 2025-10-22 19:19:45

Användare: `chef@revisionstockholm.se` (ID: NULL - ej verifierad ännu)

Program: Authentication

Rubrik: Verification Failed

Händelse: Fel verifieringskod

- IP-adress: 81.224.233.137
- Endpoint: POST `/api/auth/verify-code`
- Status: 400 Bad Request
- Fel kod: 482915 (skulle vara 482916)
- Attempts: 2/5

3. Ny kod begärd:

Datum: 2025-10-22 19:18:00

Användare: `chef@revisionstockholm.se` (ID: NULL)

Program: Authentication

Rubrik: Resend Code

Händelse: Ny verifieringskod skickad

- IP-adress: 81.224.233.137
- Endpoint: POST `/api/auth/resent-code`

- Status: 200 OK
- Ny kod: 739284
- Gamla koder raderade: 1
- Resend count: 2/3 (senaste 15 min)

4. För många försök (rate limit):

Datum: 2025-10-22 19:25:00

Användare: chef@revisionstockholm.se (ID: NULL)

Program: Security

Rubrik: Rate Limit Exceeded

Händelse: För många verifieringsförsök

- IP-adress: 81.224.233.137
- Endpoint: POST /api/auth/verify-code
- Status: 429 Too Many Requests
- Attempts: 6/5 (överskred gräns)
- Action: Blockerad tills ny kod begärs

4.7.2 CSV-filer (Mock Database)

data/aspiring_users.csv (före verifiering):

```
id,email,password_hash,created_at,expires_at
au_abc123,chef@revisionstockholm.se,$2b$12$...,2025-10-22 19:00:00,2025-10-29 19:00:00
```

data/verification_codes.csv:

```
id,email,code,expires_at,attempts,created_at
vc_482a91,chef@revisionstockholm.se,482916,2025-10-22 19:15:00,0,2025-10-22 19:00:00
```

Efter lyckad verifiering:

data/users.csv (+1 rad):

```
id,email,password_hash,role,firm_id,verified,auth_provider,created_at
550e8400,chef@revisionstockholm.se,$2b$12$...,byrachef,NULL,true,email,2025-10-22 19:25:00
```

data/refresh_tokens.csv (+1 rad):

```
id,user_id,token_hash,expires_at,created_at
rt_8f9a7b,550e8400,$2b$12$...,2025-11-21 19:20:15,2025-10-22 19:20:15
```

data/aspiring_users.csv (-1 rad, au_abc123 raderad)

data/verification_codes.csv (-1 rad, vc_482a91 raderad)

4.8 Databastabeller (Framtida PostgreSQL)

verification_codes (se Registrering Sida 2 för schema).

resent_log (ny tabell för rate limiting):

```
1 CREATE TABLE resend_log (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   email VARCHAR(255) NOT NULL,
4   requested_at TIMESTAMP NOT NULL DEFAULT NOW(),
5   ip_address INET,
6   INDEX idx_email_time (email, requested_at)
7 );
```

OBS: Cleanup-jobb raderar `resend_log` äldre än 15 minuter varje timme.

4.9 Sammanfattning

4.9.1 Scenario 1: Lyckad verifiering (happy path)

Anders Svensson registrerade sig precis (Sida 2) och ser nu Email-verifieringssidan. Han öppnar sin inbox och hittar email med kod: 482916.

Vad händer:

1. Anders matar in: 4 8 2 9 1 6 i de 6 input-fälten
2. Klickar "Verifiera"
3. Frontend anropar `POST /api/auth/verify-code`
4. Backend:
 - Validerar kod mot `verification_codes.csv` → Matchning!
 - Kontrollerar `expires_at`: 2025-10-22 19:15:00 ; NOW (19:05:00) → OK
 - Kontrollerar `attempts`: 0 ; 5 → OK
 - Läser rad från `aspiring_users.csv` (email, password_hash, id)
 - Skapar rad i `users.csv` med `verified: true`
 - Raderar rad från `aspiring_users.csv`
 - Raderar kod från `verification_codes.csv`
 - Genererar JWT tokens (accessToken 15 min, refreshToken 30 dagar)
 - Sparar refreshToken i `refresh_tokens.csv`
 - Loggar till `audit_log.csv`
 - Returnerar 200 OK med tokens
5. Frontend sparar tokens i localStorage
6. Navigerar till `/inledning` (Anders är nu inloggad!)

Backend-loggning: 1 post (Email Verified).

CSV-filer påverkade:

- `aspiring_users.csv` — -1 rad (flyttad till `users.csv`)
- `verification_codes.csv` — -1 rad (kod använd och raderad)
- `users.csv` — +1 rad (Anders är nu verified!)
- `refresh_tokens.csv` — +1 rad
- `audit_log.csv` — +1 rad

4.9.2 Scenario 2: Koden gått ut, begär ny kod

Anders glömde verifiera och kommer tillbaka 20 minuter senare. Koden har gått ut (15 min expiry).

Vad händer:

1. Anders matar in den gamla koden (482916)
2. Klickar "Verifiera"
3. Frontend anropar `POST /api/auth/verify-code`
4. Backend:
 - Validerar `expires_at`: 2025-10-22 19:15:00 ; NOW (19:25:00) → EXPIRED!
 - Returnerar 410 Gone: "Verifieringskoden har gått ut"

5. Frontend visar felmeddelande: "Koden har gått ut. Klicka 'Skicka ny kod'."
6. Anders klickar Skicka ny kod"
7. Frontend anropar `POST /api/auth/resent-code`
8. Backend:

- Kontrollerar rate limit: Räknar resend-requests senaste 15 min $\rightarrow 0$ (OK)
- Raderar gamla koder från `verification_codes.csv`
- Genererar ny kod: 739284
- Sparar i `verification_codes.csv` med `expires_at = NOW + 15 min`
- Skickar email via SendGrid: Din nya kod är: 739284"
- Loggar till `audit_log.csv`
- Returnerar 200 OK

9. Anders får email, matar in 739284, verifieras framgångsrikt!

Backend-loggning: 3 poster (Code Expired, Resend Code, Email Verified).
CSV-filer påverkade:

- `verification_codes.csv` — -1 rad (gammal kod), +1 rad (ny kod), -1 rad (använd) = -1 total
- `aspiring_users.csv` — -1 rad (flyttad till `users.csv` efter success)
- `users.csv` — +1 rad
- `audit_log.csv` — +3 rader

4.9.3 Scenario 3: För många felaktiga försök

Anders minns fel och provar olika kombinationer av koden.

Vad händer:

1. Försök 1: 482915 $\rightarrow$ 400 Fel kod" (attempts: 1/5)
2. Försök 2: 482917 $\rightarrow$ 400 Fel kod" (attempts: 2/5)
3. Försök 3: 482916 $\rightarrow$ 200 OK! (rätt kod)

Om Anders hade fortsatt gissa fel:

1. Försök 4: 482918 $\rightarrow$ 400 (attempts: 4/5)
2. Försök 5: 482919 $\rightarrow$ 400 (attempts: 5/5)
3. Försök 6: 482920 $\rightarrow$ 429 För många försök. Begär ny kod."
4. Anders måste klicka Skicka ny kod för att fortsätta

Backend-loggning: 6 poster (5 $\times$ Verification Failed, 1 $\times$ Rate Limit Exceeded).

4.9.4 Teknisk sammanfattning

- **CSV-operation:** KOPIERA `aspiring_users` $\rightarrow$ `users`, RADERA från `aspiring_users`
- **2 API endpoints:** `verify-code` (verifierar), `resent-code` (ny kod)
- **Rate limiting:** Max 5 försök per kod, Max 3 resend per 15 min
- **Kod-expiry:** 15 minuter från generering
- **JWT tokens:** Utfärdas direkt vid verifiering (auto-login)
- **Logging:** Alla försök (lyckade + misslyckade) loggas till `audit_log.csv`
- **Frontend navigation:** `/verify` $\rightarrow$ `/inledning` (vid success)
- **CSV-cleanup:** Gamla verifieringskoder raderas vid resend eller success

5 Login (Sida 4)

5.1 API-endpoints Sammanfattning

- POST `/api/auth/login` — Logga in med email/lösenord, returnera JWT tokens
- POST `/api/auth/google` — Google OAuth login/registrering (samma som Registrering)
- **Antal anrop vid inloggning:** 1 (antingen `/login` eller `/google`)
- **Rate limiting:** 5 försök per minut per IP-adress (brute force-skydd)
- **Externa integrationer:** Google OAuth API

5.2 Översikt

Inloggningssidan låter användare logga in med e-post och lösenord, eller via Google OAuth. Sidan innehåller även länk till “Glömt lösenord?” som leder till lösenordsåterställning. Vid lyckad inloggning får användaren JWT-tokens som sparas i `localStorage`.

5.3 Funktionalitet

- Användaren anger e-post och lösenord
- Vid “Logga in” görs POST till `/api/auth/login`
- Alternativt klickar användaren på “Fortsätt med Google” (OAuth-flöde)
- Vid lyckad inloggning:
 - Backend returnerar `accessToken` (15 min) och `refreshToken` (30 dagar)
 - Frontend sparar tokens i `localStorage`
 - Användaren navigeras till `/inledning` (första slide med sidebar)
- Vid fel visas felmeddelande (“Fel e-post eller lösenord”, “Kontot är inte verifierat”)
- Länk “Glömt lösenord?” navigerar till `/forgot-password`

5.4 Endpoint 1: POST `/api/auth/login`

5.4.1 Beskrivning

5.4.2 Beskrivning

Loggar in användare med e-post och lösenord. Vid lyckad inloggning returneras JWT-tokens som ger åtkomst till onboarding-flödet.

5.4.3 Request

```
1 {  
2   "email": "user@example.com",  
3   "password": "MySecurePass123"  
4 }
```

Validering:

- E-post finns i databasen (annars 401 `invalid_credentials`)
- Lösenord matchar bcrypt-hash (annars 401 `invalid_credentials`)
- Användaren är verifierad (`verified = true`, annars 403 `unverified_email`)

5.4.4 Response - Success (200 OK)

```
1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "user": {
5     "id": "uuid-123",
6     "email": "user@example.com",
7     "role": "user",
8     "firmId": "firm-uuid-456"
9   }
10 }
```

5.4.5 Response - Success (200 OK)

```
1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "user": {
5     "id": "uuid-123",
6     "email": "user@example.com",
7     "role": "user",
8     "firmId": "firm-uuid-456"
9   }
10 }
```

5.4.6 Response - Error

400 Bad Request - Saknade fält:

```
1 {
2   "error": "missing_fields",
3   "message": "Email_and_password_required"
4 }
```

401 Unauthorized - Felaktiga uppgifter:

```
1 {
2   "error": "invalid_credentials",
3   "message": "Incorrect_email_or_password"
4 }
```

403 Forbidden - Overifierad email:

```
1 {
2   "error": "unverified_email",
3   "message": "Please_verify_your_email_first"
4 }
```

429 Too Many Requests - Rate limit:

```

1 {
2   "error": "rate_limit_exceeded",
3   "message": "Too many login attempts. Try again in 1 minute"
4 }

```

5.4.7 Implementation Notes

5.4.8 Implementation Notes

- Använd `bcrypt.compare()` för lösenordskontroll
- Generera JWT med `jwt.sign()` (HS256 algoritm)
- Spara `refreshToken` i `refresh_tokens` tabell med `expires_at`
- Rate limiting: 5 försök per minut per IP-adress (förhindra brute force)
- Logga misslyckade inloggningsförsök för säkerhet
- Frontend sparar tokens i `localStorage` och navigerar till `/inledning`

5.4.9 Databastabeller

Query 1 - Hämta användare:

```

SELECT id, email, password_hash, verified, role, firm_id
FROM users
WHERE email = ?;

```

Query 2 - Spara refresh token:

```

INSERT INTO refresh_tokens (user_id, token, expires_at)
VALUES (?, ?, NOW() + INTERVAL '30 days');

```

5.5 Endpoint 2: POST `/api/auth/google`

5.5.1 Beskrivning

5.5.2 Beskrivning

Loggar in eller registrerar användare via Google OAuth. Denna endpoint används både vid registrering (Sida 2) och inloggning (Sida 4) eftersom Google OAuth hanterar båda fallen automatiskt.

OBS: Samma implementation som i Registrering (Sida 2). Se den sektionen för fullständig dokumentation.

5.5.3 Request

```

1 {
2   "code": "4/OAY0e-g7X...",
3   "redirectUri": "http://localhost:5173/login"
4 }

```

Flöde:

1. Frontend öppnar Google OAuth popup med `client_id`

2. Användaren loggar in och godkänner åtkomst
3. Google returnerar code till frontend
4. Frontend skickar code till backend
5. Backend byter code mot access_token via Google API
6. Backend hämtar användarinfo från Google (email, name, picture)
7. Om användaren finns: Logga in (returnera JWT tokens)
8. Om användaren INTE finns: Skapa ny användare med verified=true och auth_provider='google'

5.5.4 Response - Success (200 OK för login, 201 Created för ny användare)

```

1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "user": {
5     "id": "uuid-123",
6     "email": "user@gmail.com",
7     "name": "John_Doe",
8     "verified": true,
9     "authProvider": "google"
10  },
11  "isNewUser": false
12 }
```

5.5.5 Response - Success (200 OK för login, 201 Created för ny användare)

```

1 {
2   "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
4   "user": {
5     "id": "uuid-123",
6     "email": "user@gmail.com",
7     "name": "John_Doe",
8     "verified": true,
9     "authProvider": "google"
10  },
11  "isNewUser": false
12 }
```

5.5.6 Response - Error

400 Bad Request - Ogiltig kod:

```

1 {
2   "error": "invalid_code",
3   "message": "Invalid_or_expired_authorization_code"
4 }
```

503 Service Unavailable - Google API-fel:

```

1 {
2   "error": "google_api_error",
3   "message": "Google_authentication_temporarily_unavailable"
4 }

```

5.5.7 Implementation Notes

- Validera OAuth code mot Google Token API
- Hämta user info (email, name, picture) från Google
- Kontrollera om user finns i databas (sök på email)
- Om finns: Logga in, generera JWT tokens, returnera `isNewUser: false`
- Om ny: Skapa user med `verified: true, auth_provider: 'google', password_hash: null`
- Generera access token (15 min) och refresh token (30 dagar)
- Spara refresh token i `refresh_tokens` tabell
- Frontend sparar tokens i `localStorage` och navigerar till `/inledning`

5.5.8 Externa API-anrop

Google Token Exchange:

POST `https://oauth2.googleapis.com/token`

```

{
  "code": "...",
  "client_id": "...",
  "client_secret": "...",
  "redirect_uri": "...",
  "grant_type": "authorization_code"
}

```

Google User Info:

GET `https://www.googleapis.com/oauth2/v2/userinfo`

Headers: { Authorization: "Bearer <access_token>" }

5.6 Frontend - Datalagring i Browser

5.6.1 LocalStorage

Login-sidan SKRIVER till `localStorage` VID LYCKAD INLOGGNING:

Efter POST `/api/auth/login` eller POST `/api/auth/google` (200 OK):

- `accessToken` — JWT access token (15 min expiry)
- `refreshToken` — JWT refresh token (30 dagar expiry)

Före inloggning:

- Email och lösenord sparas INTE i `localStorage` (endast temporärt i React state)
- Ingen data skrivs förrän användaren har autentiserats

Logik:

- Vid lyckad inloggning: `localStorage.setItem('accessToken', data.accessToken)`
- Vid lyckad inloggning: `localStorage.setItem('refreshToken', data.refreshToken)`
- Vid fel: Ingen data skrivs, felmeddelande visas
- Efter tokens sparats: Navigera till `/inledning` (första onboarding-slide)

5.6.2 Cookies

Inga cookies sätts på denna sida. JWT tokens lagras i `localStorage` enligt vår JWT-only strategi (se Teori-sektionen).

5.7 Backend - Behandlingshistorik och GDPR-loggning

5.7.1 Loggade händelser

1. Lyckad inloggning (Email/Password):

Datum: 2025-10-22 20:15:30

Användare: Anders Svensson (ID: 550e8400-e29b-41d4-a716-446655440000)

Program: Authentication

Rubrik: Login Success

Händelse: Användaren loggade in med email/password

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: POST /api/auth/login
- Status: 200 OK
- JWT tokens utfärdade (access: 15 min, refresh: 30 dagar)

2. Lyckad inloggning (Google OAuth):

Datum: 2025-10-22 20:18:45

Användare: Lisa Karlsson (ID: 660e8400-e29b-41d4-a716-446655440001)

Program: Authentication

Rubrik: Google OAuth Login

Händelse: Användaren loggade in via Google OAuth

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...
- Endpoint: POST /api/auth/google
- Status: 200 OK
- Google email: lisa.karlsson@gmail.com
- isNewUser: false (befintlig användare)
- JWT tokens utfärdade

3. Misslyckad inloggning (Fel lösenord):

Datum: 2025-10-22 20:10:12

Användare: anders@revisionstockholm.se (ID: NULL - login misslyckades)

Program: Authentication

Rubrik: Login Failed

Händelse: Misslyckad inloggning - Fel lösenord

- IP-adress: 81.224.233.137
- User-Agent: Mozilla/5.0 (X11; Linux x86_64)...

- Endpoint: POST /api/auth/login
- Status: 401 Unauthorized
- Feltyp: invalid_credentials
- Försök: 1/5 (rate limiting räknare)

4. Misslyckad inloggning (Overifierad email):

Datum: 2025-10-22 20:12:30
 Användare: nyregistrerad@example.com (ID: 770e8400-... - ej verifierad)
 Program: Authentication
 Rubrik: Login Blocked
 Händelse: Inloggning nekad - Email inte verifierad

- IP-adress: 81.224.233.137
- Endpoint: POST /api/auth/login
- Status: 403 Forbidden
- Feltyp: unverified_email
- verified: false

5. Rate Limit Exceeded:

Datum: 2025-10-22 20:14:55
 Användare: Anonym (IP: 81.224.233.137)
 Program: Authentication
 Rubrik: Rate Limit Exceeded
 Händelse: För många inloggningsförsök från samma IP

- IP-adress: 81.224.233.137
- Endpoint: POST /api/auth/login
- Status: 429 Too Many Requests
- Försök: 6/5 (överskred gräns)
- Retry after: 60 sekunder

OBS: Misslyckade inloggningsförsök loggas för säkerhetsanalys. Detta hjälper upptäcka brute force-attacker och misstänkta inloggningsmönster.

5.7.2 CSV-filer (Mock Database)

data/audit_log.csv:

```
timestamp,user_id,user_name,program,event_type,event_details,ip_address,user_agent
2025-10-22 20:15:30,550e8400-e29b-41d4-a716-446655440000,"Anders Svensson",
  Authentication,"Login Success","Användaren loggade in med email/password -
  POST /api/auth/login - 200 OK - JWT tokens utfärdade",81.224.233.137,
  "Mozilla/5.0 (X11; Linux x86_64)..."
2025-10-22 20:10:12,NULL,"anders@revisionstockholm.se",Authentication,
  "Login Failed","Misslyckad inloggning - Fel lösenord - 401 Unauthorized -
  Försök 1/5",81.224.233.137,"Mozilla/5.0..."
```

data/users.csv (läses vid login):

```
id,email,password_hash,role,firm_id,verified,auth_provider,created_at
550e8400-e29b-41d4-a716-446655440000,anders@revisionstockholm.se,
  $2b$12$LQv3c1yqBWVHxkd0LHakCOYz6TtxMQJqhN8/LewY5NU7bxJ3v120i,
```

```
byrachef,7,true,email,2025-10-15 14:23:10
660e8400-e29b-41d4-a716-446655440001,lisa.karlsson@gmail.com,NULL,
byramedlem,7,true,google,2025-10-18 09:45:22
```

data/refresh_tokens.csv (skrivs vid login):

```
id,user_id,token_hash,expires_at,created_at
rt_8f9a7b6c5d4e3f2a1,550e8400-e29b-41d4-a716-446655440000,
  $2b$12$a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6,
  2025-11-21 20:15:30,2025-10-22 20:15:30
rt_9g0h1i2j3k4l5m6n7,660e8400-e29b-41d4-a716-446655440001,
  $2b$12$b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6a7b8,
  2025-11-21 20:18:45,2025-10-22 20:18:45
```

data/login_attempts.csv (rate limiting):

```
ip_address,timestamp,success
81.224.233.137,2025-10-22 20:10:12,false
81.224.233.137,2025-10-22 20:11:05,false
81.224.233.137,2025-10-22 20:15:30,true
```

OBS: Rate limiting implementeras genom att räkna rader i `login_attempts.csv` för en given IP-adress inom senaste minuten. Om ≥ 5 försök $\rightarrow$ 429 Too Many Requests.

5.8 Databastabeller (Framtida PostgreSQL)

När vi migrerar från CSV till PostgreSQL kommer följande tabeller användas:

users:

```
1 CREATE TABLE users (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   email VARCHAR(255) UNIQUE NOT NULL,
4   password_hash VARCHAR(255), -- NULL for Google OAuth users
5   role VARCHAR(50) DEFAULT 'user',
6   firm_id UUID REFERENCES firms(id),
7   verified BOOLEAN DEFAULT FALSE,
8   auth_provider VARCHAR(50) DEFAULT 'email', -- 'email' or '
      google'
9   created_at TIMESTAMP DEFAULT NOW(),
10  INDEX idx_email (email),
11  INDEX idx_firm_id (firm_id)
12 );
```

refresh_tokens:

```
1 CREATE TABLE refresh_tokens (
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3   user_id UUID REFERENCES users(id) ON DELETE CASCADE,
4   token_hash VARCHAR(255) NOT NULL,
5   expires_at TIMESTAMP NOT NULL,
6   created_at TIMESTAMP DEFAULT NOW(),
7   INDEX idx_user_id (user_id),
8   INDEX idx_expires_at (expires_at)
9 );
```

audit_log:

```
1 CREATE TABLE audit_log (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   timestamp TIMESTAMP NOT NULL DEFAULT NOW(),  
4   user_id UUID REFERENCES users(id),  
5   user_name VARCHAR(255),  
6   program VARCHAR(100), -- 'Authentication', 'Onboarding', etc.  
7   event_type VARCHAR(100), -- 'Login Success', 'Login Failed',  
   etc.  
8   event_details TEXT,  
9   ip_address INET,  
10  user_agent TEXT,  
11  INDEX idx_user_id (user_id),  
12  INDEX idx_timestamp (timestamp),  
13  INDEX idx_event_type (event_type),  
14  INDEX idx_ip_address (ip_address)  
15 );
```

login_attempts:

```
1 CREATE TABLE login_attempts (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   ip_address INET NOT NULL,  
4   email VARCHAR(255),  
5   success BOOLEAN NOT NULL,  
6   timestamp TIMESTAMP DEFAULT NOW(),  
7   INDEX idx_ip_timestamp (ip_address, timestamp),  
8   INDEX idx_email_timestamp (email, timestamp)  
9 );
```

5.9 Frontend Implementation (LoginSlide.jsx)

5.9.1 Email/Password Login

```
1 const handleLogin = async (e) => {  
2   e.preventDefault();  
3   try {  
4     const response = await fetch('http://localhost:8000/api/auth/  
       login', {  
5       method: 'POST',  
6       headers: { 'Content-Type': 'application/json' },  
7       body: JSON.stringify({ email, password })  
8     }  
9     \subsection{Email/Password Login}  
10    \begin{lstlisting}[language=javascript]  
11    const handleLogin = async (e) => {  
12      e.preventDefault();  
13      try {  
14        const response = await fetch('http://localhost:8000/api/auth/  
          login', {  
15          method: 'POST',  
            headers: { 'Content-Type': 'application/json' },  

```

```

16     body: JSON.stringify({ email, password })
17   });
18
19   if (!response.ok) {
20     const error = await response.json();
21     if (error.error === 'unverified_email') {
22       setError('Verifiera_din_e-post_forst');
23     } else {
24       setError('Fel_e-post_eller_losenord');
25     }
26     return;
27   }
28
29   const data = await response.json();
30   localStorage.setItem('accessToken', data.accessToken);
31   localStorage.setItem('refreshToken', data.refreshToken);
32   onNext(); // Navigate to /inledning
33 } catch (err) {
34   setError('Något_gick_fel._Försök_igen.');
```

5.9.2 Google OAuth Login

```

1  const handleGoogleLogin = () => {
2    const clientId = 'YOUR_GOOGLE_CLIENT_ID';
3    const redirectUri = 'http://localhost:5173/login';
4    const scope = 'email_profile';
5
6    const authUrl =
7      'https://accounts.google.com/o/oauth2/v2/auth?' +
8      'client_id=${clientId}&' +
9      'redirect_uri=${redirectUri}&' +
10     'response_type=code&' +
11     'scope=${scope}';
12
13    window.location.href = authUrl;
14  };
15
16  // Efter redirect tillbaka från Google:
17  useEffect(() => {
18    const urlParams = new URLSearchParams(window.location.search);
19    const code = urlParams.get('code');
```

```

27     .then(res => res.json())
28     .then(data => {
29         localStorage.setItem('accessToken', data.accessToken);
30         localStorage.setItem('refreshToken', data.refreshToken);
31         navigate('/inledning');
32     });
33 }
34 \subsubsection{Google OAuth Login}
35 \begin{lstlisting}[language=javascript]
36 const handleGoogleLogin = () => {
37     const clientId = 'YOUR_GOOGLE_CLIENT_ID';
38     const redirectUri = 'http://localhost:5173/login';
39     const scope = 'email profile';
40
41     const authUrl =
42         'https://accounts.google.com/o/oauth2/v2/auth?' +
43         'client_id=${clientId}&' +
44         'redirect_uri=${redirectUri}&' +
45         'response_type=code&' +
46         'scope=${scope}';
47
48     window.location.href = authUrl;
49 };
50
51 // Efter redirect tillbaka från Google:
52 useEffect(() => {
53     const urlParams = new URLSearchParams(window.location.search);
54     const code = urlParams.get('code');
55
56     if (code) {
57         fetch('http://localhost:8000/api/auth/google', {
58             method: 'POST',
59             headers: { 'Content-Type': 'application/json' },
60             body: JSON.stringify({ code, redirectUri: window.location.
61                 origin + '/login' })
62         })
63         .then(res => res.json())
64         .then(data => {
65             localStorage.setItem('accessToken', data.accessToken);
66             localStorage.setItem('refreshToken', data.refreshToken);
67             navigate('/inledning');
68         });
69     }
70 }, []);

```

5.10 Sammanfattning

5.10.1 Scenario 1: Lyckad inloggning med Email/Password

Anders Svensson, byråchef på Revision Stockholm AB, har tidigare registrerat sig och verifierat sin e-post. Han besöker login-sidan och matar in sitt email (anders@revisionstockholm.se)

och lösenord.

Steg-för-steg:

1. Anders klickar Logga in" → Frontend skickar POST /api/auth/login med email + lösenord
2. Backend:
 - Läser `users.csv`, hittar Anders användare med `verified=true`
 - Verifierar lösenord med `bcrypt (bcrypt.compare())`
 - Lösenord matchar → Generera JWT tokens (access 15 min, refresh 30 dagar)
 - Spara refresh token hash i `refresh_tokens.csv`
 - Logga händelse till `audit_log.csv`: Login Success"
3. Backend returnerar: 200 OK med `accessToken`, `refreshToken`, `user` object
4. Frontend:
 - Sparar tokens: `localStorage.setItem('accessToken', ...)`
 - Sparar tokens: `localStorage.setItem('refreshToken', ...)`
 - Navigerar till /inledning (första onboarding-slide med sidebar)

Resultat: Anders är nu inloggad och kan fortsätta med onboarding-flödet. Hans session är aktiv i 15 minuter (access token), därefter förnyas automatiskt via refresh token (giltigt i 30 dagar).

5.10.2 Scenario 2: Lyckad inloggning med Google OAuth

Lisa Karlsson är ny användare som vill registrera sig via Google. Hon klickar Fortsätt med Google på login-sidan.

Steg-för-steg:

1. Lisa klickar Fortsätt med Google" → Redirectas till Google OAuth consent screen
2. Lisa godkänner åtkomst → Google returnerar `code` till frontend
3. Frontend skickar POST /api/auth/google med `code` + `redirectUri`
4. Backend:
 - Byter `code` mot `access_token` via Google Token API
 - Hämtar Lisa's email från Google User Info API: `lisa.karlsson@gmail.com`
 - Kollar `users.csv` → Lisa finns INTE (ny användare)
 - Skapar ny rad i `users.csv`: `verified=true, auth_provider='google', password_hash=NULL`
 - Genererar JWT tokens (access 15 min, refresh 30 dagar)
 - Sparar refresh token hash i `refresh_tokens.csv`
 - Loggar händelse: Google OAuth Login - `isNewUser: true`"
5. Backend returnerar: 201 Created med tokens + `isNewUser: true`
6. Frontend:
 - Sparar tokens i `localStorage`
 - Navigerar till /inledning (hoppas över email-verifiering, Google-användare är auto-verifierade)

Resultat: Lisa skapas som ny användare OCH loggas in i samma steg. Hon behöver inte verifiera email eftersom Google redan garanterat ägandeskap av emailadressen.

5.10.3 Scenario 3: Misslyckad inloggning (Fel lösenord)

Anders försöker logga in men skriver fel lösenord.

Steg-för-steg:

1. Anders matar in email + FEL lösenord → Klickar Logga in”
2. Frontend skickar POST /api/auth/login
3. Backend:
 - Läser `users.csv`, hittar Anders användare
 - Verifierar lösenord med `bcrypt` → `MATCH` INTE
 - Loggar händelse till `audit_log.csv`: Login Failed - Fel lösenord - Försök 1/5”
 - Skriver rad till `login_attempts.csv`: `ip_address, timestamp, success=false`
 - Kollar rate limiting: Antal misslyckade försök senaste minuten = 1 (OK, i 5)
4. Backend returnerar: 401 Unauthorized {"error": "invalid_credentials"}
5. Frontend:
 - Visar felmeddelande: Fel e-post eller lösenord”
 - Anders kan försöka igen (har 4 försök kvar innan rate limit)

OBS: Om Anders gör 5 misslyckade försök inom 1 minut → 429 Too Many Requests, måste vänta 60 sekunder.

5.10.4 Scenario 4: Inloggning nekad (Overifierad email)

En ny användare (Lisa) registrerade sig igår men verifierade aldrig sin email. Hon försöker nu logga in.

Steg-för-steg:

1. Lisa matar in email + lösenord → Klickar Logga in”
2. Frontend skickar POST /api/auth/login
3. Backend:
 - Läser `users.csv`, hittar Lisa’s användare med `verified=false`
 - Lösenord matchar, MEN användaren är inte verifierad
 - Loggar händelse: Login Blocked - Email inte verifierad”
4. Backend returnerar: 403 Forbidden {"error": "unverified_email"}
5. Frontend:
 - Visar meddelande: ”Verifiera din e-post först. Kolla din inkorg för verifieringskoden.”
 - Länk till /verify för att mata in kod

Resultat: Lisa måste slutföra email-verifieringen innan hon kan logga in.

5.10.5 Teknisk sammanfattning

- **2 API endpoints:** POST /api/auth/login (email/password), POST /api/auth/google (OAuth)
- **Rate limiting:** 5 försök per minut per IP-adress (brute force-skydd via `login_attempts.csv`)
- **JWT tokens:** accessToken (15 min), refreshToken (30 dagar) sparas i **localStorage** (INTE cookies)

- **CSV-filer:** Läser `users.csv`, skriver till `refresh_tokens.csv`, `audit_log.csv`, `login_attempts.csv`
- **Google OAuth:** Auto-verifierade användare (`verified=true`), ingen email-verifiering behövs, `password_hash=NULL`
- **Säkerhet:** bcrypt password hashing, JWT signature verification, rate limiting, audit logging
- **Navigation:** Lyckad login → `/inledning` (första slide med sidebar)
- **Error handling:** 3 error types (`invalid_credentials`, `unverified_email`, `rate_limit_exceeded`)
- **Glömt lösenord:** Länk till `/forgot-password` (dokumenteras i Sida 5)
- **GDPR logging:** Alla login-försök loggas med IP-adress, timestamp, user-agent (Fortnox format)

6 Intro (Slide 5)

TODO: Dokumentera intro-endpoints här (första slide med sidebar)...

7 Uppdrag (Slide 6)

TODO: Dokumentera uppdrag-endpoints här...

8 Verksamhet (Slide 7)

TODO: Dokumentera verksamhet-endpoints här...

9 Riskfrågor (Slide 8)

TODO: Dokumentera riskfrågor-endpoints här...

10 Identitetskontroll (Slide 9)

TODO: Dokumentera BankID-endpoints här...

11 Kontrolltabell (Slide 10)

TODO: Dokumentera kontrolltabell-endpoints här...

12 PEP-fördjupning (Slide 11)

TODO: Dokumentera PEP-endpoints här...

13 Verksamhetskod (Slide 12)

TODO: Dokumentera SNI-kod-endpoints här...

14 Ägarstruktur (Slide 13)

TODO: Dokumentera Bolagsverket-endpoints här...

15 Styrelse (Slide 14)

TODO: Dokumentera styrelse-endpoints här...

16 Företagsdokumentation (Slide 13.5)

TODO: Dokumentera PDF-upload och OCR-endpoints här...

17 Bokföringsunderlag (Slide 14.5)

TODO: Dokumentera file-upload och fraud-detection-endpoints här...

18 Bokföring (Slide 15)

TODO: Dokumentera Skatteverket/SIE-endpoints här...

19 Likviditet (Slide 16)

TODO: Dokumentera likviditets-analys-endpoints här...

20 Omsättning (Slide 17)

TODO: Dokumentera omsättnings-data-endpoints här...

21 Resultat (Slide 18)

TODO: Dokumentera resultat-analys-endpoints här...

22 Bransch (Slide 19)

TODO: Dokumentera bransch-jämförelse-endpoints här...

23 Bokföringsanalys (Slide 20)

TODO: Dokumentera AI-analys-endpoints här...

24 Riskbesked (Slide 21)

TODO: Dokumentera risk-scoring-endpoints här...

25 Skyldigheter (Slide 22)

TODO: Dokumentera avtal-endpoints här...

26 Fortnox-koppling (Slide 23)

TODO: Dokumentera Fortnox OAuth-endpoints här...

27 Bankrättigheter (Slide 24)

TODO: Dokumentera bank-OAuth-endpoints här...

28 AdminDashboard

TODO: Dokumentera admin-endpoints här (referera till Admin_Dashboard_Spec.tex)...

29 Appendix

29.1 Gemensamma Error Codes

Code	Status	Beskrivning
400	Bad Request	Ogiltig input data
401	Unauthorized	Saknar eller ogiltig JWT token
403	Forbidden	Saknar behörighet för resursen
404	Not Found	Resursen finns inte
409	Conflict	Resurs existerar redan (t.ex. email)
422	Unprocessable Entity	Valideringsfel
429	Too Many Requests	Rate limit överskriden
500	Internal Server Error	Oväntat serverfel
503	Service Unavailable	Extern tjänst ej tillgänglig

29.2 Rate Limiting

- **Auth endpoints:** 5 requests/minut per IP
- **File uploads:** 10 requests/minut per user
- **Övriga endpoints:** 100 requests/minut per user

29.3 File Upload Limits

- **Max filstorlek:** 10 MB per fil
- **Max totalt per request:** 50 MB
- **Tillåtna format:** PDF, PNG, JPG, JPEG, SIE, XML

30 Del 2: Onboarding-Slides (Med Sidebar)

Detta avsnitt dokumenterar API-endpoints för alla onboarding-slides som visas **efter** autentisering. Användaren har nu JWT tokens i localStorage och ser en sidebar med progress-indikator.

Onboarding-flödet består av cirka 20-25 slides som samlar in kundinformation enligt Penningtvättslagen (PML) och Revisorslagens (RL) krav. Varje slide sparar data i backend och användaren kan navigera fram/tillbaka med progress bevarad.

30.1 Gemensamma mönster

- **Autentisering:** Alla endpoints kräver `Authorization: Bearer <access_token>` header
- **User context:** Backend hämtar `user_id` från JWT token
- **Progress tracking:** Varje slide-completion sparas i `onboarding_progress.csv`
- **Draft saves:** Data sparas som drafts vid varje fältändring (debounced 2 sekunder)
- **Validation:** Frontend validerar innan POST, backend validerar igen
- **Error handling:** 400 (validation), 401 (auth), 500 (server error)

30.2 Sida 6: Inledning och bakgrund

30.2.1 API-endpoints Sammanfattning

INGA API-endpoints – Denna slide är rent informativ och kräver ingen data från backend.

30.2.2 Översikt

Första sliden efter autentisering. Visar information om varför onboarding-processen är nödvändig (Penningtvättslagen, tillsyn av Länsstyrelserna, GDPR-krav). Innehåller endast en Nästa-knapp.

React-komponent: IntroSlide.jsx

Route: /inledning

User journey:

1. Användaren loggar in → Navigeras till /inledning
2. Läser informationen om PML-krav och Länsstyrelsens tillsyn
3. Klickar Nästa → Navigeras till /riskfragor

Sidebar: Visar "1. Inledningsom aktiv, övriga slides som grå

30.2.3 Frontend - Datalagring i Browser

- **localStorage:** accessToken, refreshToken (redan sparade från login)
- **sessionStorage:** INGET (behövs ej)
- **State:** Ingen slide-specifik state (bara navigation)

30.2.4 Backend - GDPR-loggning

- **Event:** "Önboarding Started" (när användaren landar på /inledning första gången)
- **Detaljer:** slide: "intro", timestamp: "2025-10-22 20:30:15"
- **Sparas i:** audit_log.csv

Exempel audit_log post:

```
1 2025-10-22 20:30:15,550e8400,anders@gmail.com,Onboarding ,
   Onboarding Started ,
2 {"slide":"intro"},192.168.1.10,Mozilla/5.0...
```

OBS: Denna loggning sker automatiskt när React-appen navigerar till /inledning. Inget explicit API-anrop från frontend.

30.2.5 CSV-filer

INGA nya CSV-filer – IntroSlide sparar ingen data.

Befintliga filer som kan läsas:

- **users.csv** – Backend kan läsa user_name för att visa "Välkommen, Anders!" (valfritt)

30.2.6 Databastabeller

onboarding_progress: (PostgreSQL)

```
1 CREATE TABLE onboarding_progress (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   user_id UUID NOT NULL REFERENCES users(id),  
4   current_slide VARCHAR(50) NOT NULL, -- 'intro', 'riskfragor',  
    'foretagsinfo', etc.  
5   completed_slides TEXT[], -- Array: ['intro', 'riskfragor']  
6   last_updated TIMESTAMP NOT NULL DEFAULT NOW(),  
7   INDEX idx_user_id (user_id)  
8 );
```

OBS: När användaren klickar Nästafrån IntroSlide, uppdatera:

```
1 UPDATE onboarding_progress  
2 SET current_slide = 'riskfragor',  
3     completed_slides = array_append(completed_slides, 'intro'),  
4     last_updated = NOW()  
5 WHERE user_id = '<user_id_from_jwt>';
```

30.2.7 Sammanfattning

IntroSlide är en **läs-onlyslide** utan API-anrop från frontend. Backend kan dock logga när användaren når denna slide första gången (via middleware som kollar route changes) för GDPR-compliance och progress tracking.

Teknisk sammanfattning:

- Ingen data input från användaren
- Ingen data sparad i backend från denna slide
- Progress tracking: "introläggs till i `completed_slides[]` när Nästa" klickas
- Navigation: Klick "Nästa" → Frontend navigerar till `/riskfragor`
- Sidebar uppdatering: React state management (ingen backend sync)